

# **Trabajo Práctico Integrador**

**Universidad Técnica Nacional**

**Carrera:** Tecnicatura Universitaria en Programación.

**Materia:** Programación II.

**Proyecto:** Sistema de Gestión de pedidos de Comida.

**Alumno:** Francisco Bultynch.

**Grupo:** 70

**Fecha:** 22/06/2026

## **Índice**

**2** - Introducción.

**2** - Objetivos.

**2** - Marco teórico.

**3** - Diagrama UML.

**4** - Arquitectura del proyecto.

**5** - Descripción de las entidades.

**5** - Servicios implementados.

**6** - Funcionalidades implementadas.

**7** - Manejo de excepciones.

**7** - Conclusiones.

**7** - Bibliografía.

## Introducción

El presente Trabajo Práctico Integrador fue desarrollado utilizando Java y aplicando los conceptos estudiados durante la materia Programación II. El objetivo principal fue construir un sistema de gestión de pedidos que permita administrar categorías, productos, usuarios y pedidos mediante operaciones CRUD y aplicando los principios de Programación Orientada a Objetos.

## Objetivos

- Objetivo general:

Desarrollar una aplicación de consola en Java aplicando los conceptos fundamentales de POO.

- Objetivos específicos:

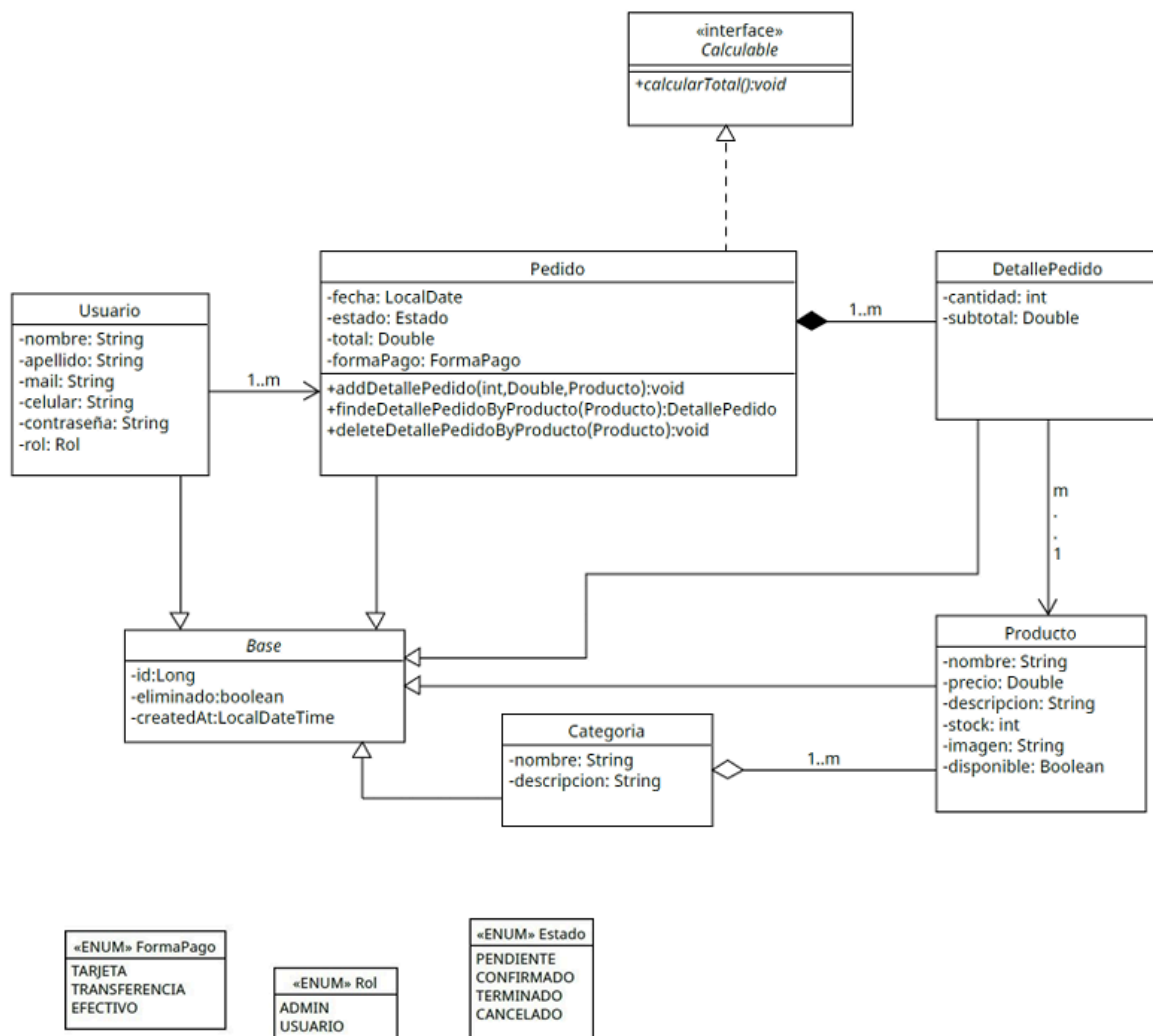
- Implementar herencia.
- Utilizar interfaces.
- Emplear enumeraciones.
- Utilizar colecciones.
- Implementar excepciones personalizadas.
- Desarrollar operaciones CRUD.
- Aplicar encapsulamiento y composición entre clases.

## Marco teórico

- POO: la Programación Orientada a Objetos permite modelar problemas del mundo real mediante clases y objetos. Los principales conceptos utilizados fueron encapsulamiento, herencia y composición.
- Herencia: se implementó una clase base desde la cual heredan las entidades principales del sistema.
- Interfaces: se utilizó una interfaz dominada calculable encargada de definir el método `calcularTotal()` utilizado en los pedidos.
- Enumeraciones: se implementaron enum para representar el estado del pedido, la forma de pago y el rol del usuario.

- Colecciones: se utilizaron ArrayList para almacenar las distintas entidades del sistema.
- Excepciones: se implementaron excepciones personalizadas para controlar situaciones de error.

## Diagrama UML



El diseño del sistema fue realizado a partir del diagrama UML provisto en la consigna. En él pueden observarse las relaciones entre las entidades, la herencia y la implementación de la interfaz calculable.

## Arquitectura del proyecto

El proyecto fue dividido en distintos paquetes:

- entities: contiene las clases principales del sistema
  - base
  - categoria
  - producto
  - usuario
  - pedido
  - detallePedido
- services: contiene la logica de negocio
  - categoriaService
  - productoService
  - usuarioService
  - pedidoService
- enums:
  - estado
  - formaPago
  - rol
- exception:
  - entidadNoEncontradaException
  - stockInvalidoException
  - emailDuplicadoException
- interfaces:
  - calculable
- menu:
  - menuPrincipal

## Descripción de las entidades

- Categoría: representa las categorías de los productos y contiene nombre y descripción.
- Producto: representa los productos disponibles para la venta, incluyendo precio, stock y categoría.
- Usuario: representa a los clientes del sistema.
- DetallePedido: representa cada producto incluido en un pedido junto con la cantidad y subtotal.

## Servicios implementados

Se desarrollaron servicios para administrar las distintas entidades.

- CategoriaService
  - Permite:
    - Crear categorías.
    - Busca categorías.
    - Modificar categorías.
    - Eliminar categorías.
- ProductoService:
  - Permite:
    - Crear productos.
    - Busca productos.
    - Modificar productos.
    - Eliminar productos.
- UsuarioService:
  - Permite:
    - Crear usuarios.
    - Busca usuarios.
    - Modificar usuarios.
    - Eliminar usuarios.

- PedidoService:
  - Permite:
    - Crear pedidos.
    - Actualizar estado.
    - Actualizar la forma de pago.
    - Eliminar pedidos.

## **Funcionalidades implementadas**

- Categorías:
  - Crear.
  - Listar.
  - Editar.
  - Eliminar.
- Productos:
  - Crear.
  - Listar.
  - Editar.
  - Eliminar.
- Usuarios:
  - Crear.
  - Listar.
  - Editar.
  - Eliminar.
- Pedidos:
  - Crear.
  - Agregar varios productos
  - Calcular el total.
  - Actualizar estado.
  - Actualizar la forma de pago.
  - Eliminar.

## **Manejo de excepciones**

Se implementaron excepciones personalizadas para mejorar el control de errores:

- entidadNoEncontradaExcepcion: se utiliza cuando se intenta acceder a una entidad inexistente.
- stockInvalidoExcepcion: se utiliza cuando el stock ingresado es válido.
- emailDuplicadoExcepcion: se utiliza cuando se intenta registrar un correo electrónico ya existente.

## **Conclusiones**

El desarrollo del Trabajo Práctico Integrador permitió aplicar los conocimientos adquiridos durante la cursada, especialmente los relacionados con Programación Orientada a Objetos, colecciones, interfaces y excepciones. Además, permitió comprender la importancia de la organización del código y de la separación entre entidades, servicios y presentación.

## **Bibliografía**

- Oracle. Java Documentation.
- Deitel & Deitel. Java: Cómo Programar.
- Material teórico de Programación II.
- Apuntes de la cátedra.
- Documentación oficial de Java Collections Framework.